Python 金融数据分析

第 3 章　*NumPy* 数组计算

主讲人：　王伟冠, 吕文俊
院　系：　经济学院金融系

上海大学

2026 年 10 月 14 日



1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. 随机数与重复模拟
8. 综合案例与小结

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. 随机数与重复模拟
8. 综合案例与小结

- 理解为什么金融数据分析离不开 NumPy
- 掌握 `ndarray` 的创建、属性与索引切片
- 掌握**向量化运算**：一行代码对整个数组做计算
- 掌握**布尔索引**：按条件筛选数据
- 掌握常用统计函数（均值、标准差、累计等）
- 能够进行排序、缺失值检查与加权计算
- 能用随机数和二维数组重复模拟，解释频率与概率的区别

前置知识

第 2 章的列表、循环、函数——本章会把它们"升级"为数组写法。

```python
prices = [100.0, 102.0, 101.0, 105.0]
returns = []
for i in range(1, len(prices)):
    r = (prices[i] - prices[i-1]) / prices[i-1]
    returns.append(r)
```

- 只有 4 个价格没问题；但真实数据动辄数千个交易日、成百上千只股票。
- Python 的 `for` 循环每一步都要检查类型、逐个搬运，**速度慢**；
- 代码冗长，意图（"算收益率"）被循环细节淹没。

本章的目标

用 NumPy 把 4 行循环压缩成 1 行，并且快上百倍。

NumPy 是什么?

- **NumPy**（Numerical Python）是 Python 科学计算的基石。
- 核心对象：ndarray——存储**同一类型**数据的多维数组。
- 底层用 C 实现，运算直接作用于整块内存，速度远超 Python 循环。
- pandas、scikit-learn 等后续工具都构建在 NumPy 之上。

```python
import numpy as np      # 约定俗成的导入方式

prices = np.array([100.0, 102.0, 101.0, 105.0])
returns = prices[1:] / prices[:-1] - 1    # 一行搞定!
```

约定

`import numpy as np` 是全世界通用的写法，请照抄。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. 随机数与重复模拟
8. 综合案例与小结

```
a = np.array([100.0, 102.0, 101.0, 105.0])
a             # array([100., 102., 101., 105.])

type(a)       # numpy.ndarray
a.shape       # (4,)      形状：长度为4的一维数组
a.dtype       # float64   元素类型：全数组统一
len(a)        # 4
```

- np.array(列表)：把 Python 列表转换为数组。
- **类型统一**：数组中所有元素同一类型（如全是 float64），这正是快的原因。
- shape 描述" 形状"，是后面理解二维数据的关键。

快速生成常用数组

```
np.zeros(5)             # array([0., 0., 0., 0., 0.])
np.ones(3)              # array([1., 1., 1.])

np.arange(0, 10, 2)     # array([0, 2, 4, 6, 8])  类似range
np.linspace(0, 1, 5)    # array([0., 0.25, 0.5, 0.75, 1.])
                        # 0到1之间均匀取5个点（含端点）
```

- zeros/ones：初始化占位数组（如预存结果）。
- arange(起，止，步长)：整数序列，与 range 类似但返回数组。
- linspace(起，止，个数)：画利率区间、画函数图像时常用。

二维数组：用股票价格理解 shape 与 axis

```python
# 3天 x 2只股票 的收盘价（元）：每行一天，每列一只股票
p = np.array([[25.0, 100.0],
              [25.5,  99.0],
              [26.0, 102.0]])

p.shape       # (3, 2)    3行2列
p.ndim        # 2         维度数
p.size        # 6         元素总数
p.mean(axis=0)  # [25.5  100.33333333]  每只股票的3日均价
p.mean(axis=1)  # [62.5  62.25  64.0]   每天两只股票的均价
```

- shape=(3, 2)：行 = 日期，列 = 股票，两个轴依次编号为 0、1。
- axis=0：沿日期方向求均值，得到 2 只股票各自的均价。
- axis=1：沿股票方向求均值，得到 3 个日期各自的均价。

每日横截面均价仅用于演示计算方向，不代表投资组合收益。

```
a = np.arange(6)                # [0 1 2 3 4 5]
a.reshape(2, 3)                 # 变成2行3列
a.reshape(-1, 2)                # -1 表示"自动算": 3行2列

np.array([1, 2]) + 0            # 数组拼接用专门函数:
np.concatenate([np.array([1, 2]), np.array([3, 4])])
                                # array([1, 2, 3, 4])
```

易错点

数组的 + 是**逐元素相加**，不是拼接！拼接请用 np.concatenate。
np.array([1,2]) + np.array([3,4]) 得到 [4, 6]。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. 随机数与重复模拟
8. 综合案例与小结

一维数组：与列表几乎相同

```
prices = np.array([30.1, 30.5, 29.8, 31.2, 30.9])

prices[0]          # 30.1     索引从0开始
prices[-1]         # 30.9     倒数第一个
prices[1:3]        # array([30.5, 29.8])    含头不含尾
prices[1:]         # 除第一个外的所有元素
prices[:-1]        # 除最后一个外的所有元素
```

- 索引、切片规则与列表完全一致——旧知识直接复用。
- `prices[1:]` 与 `prices[:-1]` 这对组合后面算收益率要用，务必熟练。

注意

数组切片返回的是**视图**（与原数组共享内存）：修改切片会影响原数组，与列表不同。

二维数组：先行后列

```
p = np.array([[25.0, 100.0],
              [25.5,  99.0],
              [26.0, 102.0]])

p[0, 0]       # 25.0   第0行第0列
p[1, 1]       # 99.0   第1行第1列
p[0]          # 第0行 -> array([25., 100.])
p[:, 1]       # 所有行、第1列 -> 股票2的价格序列
p[1:, :]      # 第1行到末尾的所有行
```

- 写法：数组 [行，列]；冒号：表示"全部"。
- p[:, 1] 即"取出某只股票的整个价格序列"，金融分析中的高频操作。

基本切片通常共享原数组的数据。需要独立修改时加 copy()。这与 Python 列表的切片不同。

```
measurements = np.array([10, 20, 30, 40])
view = measurements[1:3]
independent = measurements[1:3].copy()
view[0] = 99
print(measurements)  # [10 99 30 40]
print(independent)   # [20 30]
```

改成 measurements[[1, 2]] 后再修改结果，原数组会变化吗？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

怎样把三名同学的成绩从低到高排列，同时保留姓名对应关系？

```
names = np.array(["小林", "小周", "小王"])
marks = np.array([80, 95, 70])
order = np.argsort(marks)
print(np.sort(marks))
print(order)
print(names[order])
```

运行输出

```
[70 80 95]
[2 0 1]
['小王' '小林' '小周']
```

sort 返回排序后的值；argsort 返回原位置。用同一组位置索引姓名和成绩，配对关系不会丢失。

沿用位置索引：如何同时取出股票名称和收益率？收益率为小数，0.05 表示 5%。

```
symbols = np.array(["A", "B", "C", "D"])
stock_returns = np.array([0.03, -0.02, 0.05, 0.01])
top = np.argsort(stock_returns)[::-1][:2]
print(symbols[top])
print(stock_returns[top])
```

运行输出

```
['C' 'A']
[0.05 0.03]
```

先升序排序，再反向取前两项。这是给定样本的收益排名，不是对未来收益的预测。

练习：找到收益最低的两只股票

修改上一例，同时输出收益最低的两只股票名称和收益率。

动手检查

先预测结果，再运行验证。参考答案见 Notebook 的折叠单元格。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. **向量化运算**
5. 布尔索引：按条件筛选
6. 统计函数
7. 随机数与重复模拟
8. 综合案例与小结

```
prices = np.array([100.0, 102.0, 101.0, 105.0])

prices + 1        # 每个元素加1
prices * 1.05     # 每个元素涨5%
prices ** 2       # 每个元素平方
1 / prices        # 每个元素取倒数
```

- **向量化**：运算自动作用于数组中的每一个元素，无需循环。
- 运算不改变原数组，而是返回**新数组**。
- 读代码时把数组想象成"一整列 Excel 数据同时参与计算"。

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([10., 20., 30.])

a + b          # array([11., 22., 33.])
a * b          # array([10., 40., 90.])
b / a          # array([10., 10., 10.])
```

- 两个数组运算：**位置一一对应**（第 0 个对第 0 个、第 1 个对第 1 个……）。
- 这正对应金融场景：
  - "今日价 / 昨日价"：两个错开一位的价格序列相除；
  - "组合收益"：持仓权重数组 × 个股收益数组。

形状要匹配

参与运算的两个数组形状必须一致（或满足广播规则），否则报错。

(2, 3) 与 (3,) 可以逐列配对。广播从末尾维度比较，长度相同或其中一个为 1 才能兼容。

```
scores = np.array([[80, 90, 70], [60, 75, 90]])
bonus = np.array([2, 0, 5])
adjusted = scores + bonus
print(adjusted)
# [[82 90 75]
#  [62 75 95]]

student_mean = scores.mean(axis=1, keepdims=True)
print(student_mean.shape)  # (2, 1)
print(scores - student_mean)
# [[  0.  10. -10.]
#  [-15.   0.  15.]]
```

将每一科减去该科均分；解释结果为什么每列均值为 0。

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

例：一行代码计算收益率

```
prices = np.array([100.0, 102.0, 101.0, 105.0])

# 简单收益率：(今日-昨日)/昨日
ret = prices[1:] / prices[:-1] - 1
ret         # array([0.02, -0.0098, 0.0396])

# 对数收益率（后文常用）
logret = np.log(prices[1:] / prices[:-1])
```

- `prices[1:]`：今日价序列；`prices[:-1]`：昨日价序列；相除即逐日配对。
- 上一章的 4 行循环，现在 1 行；数据越大，优势越明显。
- `np.log()` 是向量化函数，对整个数组取自然对数。

快多少？速度对比

```python
big = np.arange(1_000_000)          # 0到999999

%timeit [x ** 2 for x in big]       # Python循环：约200毫秒
%timeit big ** 2                    # NumPy向量化：约1毫秒
```

- %timeit 是 Notebook 计时魔法命令（下节 notebook 中可亲自体验）。
- 同样的一百万元素平方运算，NumPy 快约 100 倍以上。
- 原因：类型统一 + 底层 C 实现 + 无需逐步类型检查。

经验法则

看到"对每个元素做同样的事"，先想向量化，再考虑循环。

常用向量化数学函数

函数	作用	金融用途举例
`np.log(x)` / `np.exp(x)`	对数 / 指数	对数收益率、连续复利
`np.sqrt(x)`	平方根	波动率换算
`np.abs(x)`	绝对值	偏差幅度
`np.maximum(a,b)`	逐元素取大	期权收益 $\max(S-K,0)$
`np.round(x, n)`	四舍五入	报表展示
`np.cumsum(x)`	累计和	累计收益、净值曲线
`np.diff(x)`	相邻差分	价格变动额

期权到期收益

`np.maximum(S - K, 0)`：行权价 K，到期价序列 S，一行算出所有情形的收益。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. **布尔索引：按条件筛选**
6. 统计函数
7. 随机数与重复模拟
8. 综合案例与小结

比较运算返回布尔数组

```
ret = np.array([0.02, -0.01, 0.04, -0.03, 0.01])

ret > 0       # array([True, False, True, False, True])
```

- 数组的比较运算也是**向量化**的：逐元素判断，返回等长的布尔数组。
- 布尔数组可以当"筛选器"使用（下一页）。
- 组合条件要用 `&`（且）、`|`（或），且每个条件加括号：

```
(ret > -0.01) & (ret < 0.01)      # 小幅波动日
```

易错点

数组条件中 `and`、`or` 会报错，必须用 `&`、`|`。

用布尔数组筛选与统计

```
ret = np.array([0.02, -0.01, 0.04, -0.03, 0.01])

ret[ret > 0]           # array([0.02, 0.04, 0.01]) 只留上涨日
ret[ret < -0.02]       # array([-0.03])            大跌日

np.sum(ret > 0)        # 3      True按1计数：上涨天数
(ret > 0).mean()       # 0.6    上涨日占比
```

- 数组 [布尔数组]：只保留对应位置为 True 的元素。
- np.sum(布尔数组)：统计满足条件的个数——非常实用的技巧。
- 金融应用：统计涨停天数、回撤超过阈值的次数、极端波动日。

np.where：条件赋值

```
ret = np.array([0.02, -0.01, 0.04])

np.where(ret > 0, 1, 0)
# array([1, 0, 1])   条件为真取1，否则取0

np.where(ret > 0, "涨", "跌")
# array(['涨', '跌', '涨'])
```

- np.where(条件，真时的值，假时的值)：向量化的 if/else。
- 应用：构造交易信号——收益为正记 1（持有多头），为负记 0。
- 这比写 for 循环套 if 简洁且快得多。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. **统计函数**
7. 随机数与重复模拟
8. 综合案例与小结

```
ret = np.array([0.02, -0.01, 0.04, -0.03, 0.01])

ret.sum()         # 0.03     求和
ret.mean()        # 0.006    均值（日均收益率）
ret.std()         # 0.024... 标准差（波动性）
ret.max()         # 0.04     最大值
ret.min()         # -0.03    最小值
ret.argmax()      # 2        最大值的"位置"
```

- " 把一串数压缩成一个数" 的函数叫**聚合函数**。
- 写法两种等价：`ret.mean()` 或 `np.mean(ret)`。
- **标准差**是金融中风险（波动率）的度量，务必熟悉。

二维数组与 axis 参数

```
# 3天 x 2只股票的日收益率
r = np.array([[ 0.02, -0.01],
              [-0.01,  0.03],
              [ 0.04,  0.01]])

r.mean()          # 0.0133...  全部元素的均值
r.mean(axis=0)    # 按"列"求均值 -> 每只股票的日均收益
r.mean(axis=1)    # 按"行"求均值 -> 每天的组合平均收益
```

- axis=0：沿行的方向压缩 → 结果对应每一**列**；
- axis=1：沿列的方向压缩 → 结果对应每一**行**。
- 记忆口诀：axis 是被"压掉"的那一维。

```python
prices = np.array([100., 102., 101., 105.])

np.diff(prices)      # array([2., -1., 4.]) 每日涨跌"额"
np.cumsum([1, 1, 1, 1])   # array([1, 2, 3, 4]) 累计和
```

- `np.diff`：相邻元素相减，长度少 1——快速得到每日变动。
- `np.cumsum`：逐步累加——从" 每日现金流" 得到" 累计现金流"，从" 对数收益率" 恢复" 累计对数收益"。

小例子

对数收益率序列取 `np.cumsum`，再取 `np.exp`，即得净值曲线（相对期初价格）。

一人成绩缺失时，能否把缺失当成零分？先检查缺失，再决定统计口径。

```
marks_missing = np.array([80., np.nan, 90.])
print(np.isnan(marks_missing))
print(marks_missing.mean())
print(np.nanmean(marks_missing))
```

运行输出

```
[False  True False]
nan
85.0
```

nan 表示缺失；普通 mean 得到 nan，nanmean 忽略缺失后得到 85。它描述两名有成绩的学生，不是三人的完整均分。

计算机保存小数时可能产生微小舍入误差。怎样检查计算结果是否足够接近？

```
value = 0.1 + 0.2
print(value)
print(value == 0.3)
print(np.isclose(value, 0.3, rtol=0, atol=1e-12))
```

运行输出

```
0.30000000000000004
False
True
```

isclose 比较数值是否接近。这里 rtol=0 关闭相对容差，atol=1e-12 允许极小的绝对误差；容差应随实际单位与精度要求选择。

两门课成绩为 80、90，权重分别为 40%、60%，综合成绩是多少?

```
marks_weighted = np.array([80., 90.])
weights = np.array([0.4, 0.6])
print(marks_weighted * weights)
print((marks_weighted * weights).sum())
```

运行输出

```
[32. 54.]
86.0
```

先逐项乘权重得到 32、54，再相加得到 86。权重和为 1；等权平均只是加权平均的特殊情形。

每行是一天，每列是一只股票；假设每天期初权重均为 40%、60%，忽略交易成本。

```
asset_returns = np.array([[0.02, -0.01],
                          [-0.01, 0.03],
                          [0.04, 0.01]])
weights = np.array([0.4, 0.6])
print((asset_returns * weights).sum(axis=1))
print(asset_returns @ weights)
```

运行输出

```
[0.002 0.014 0.022]
[0.002 0.014 0.022]
```

第一天为 $0.02\times0.4-0.01\times0.6=0.002$，即 0.2%。* 逐元素相乘；@ 在这里把每行与权重相乘再求和：(3, 2) 与 (2,) 得到 (3,)。

练习：改变持仓权重

把两只股票的权重改成 70%、30%，计算三天的组合收益率，手算核对第一天。

动手检查

先预测结果，再运行验证。参考答案见 Notebook 的折叠单元格。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. **随机数与重复模拟**
8. 综合案例与小结

怎样模拟掷六面骰子？先创建生成器，再指定整数范围和抽样数量。

```
dice_rng = np.random.default_rng(7)
dice = dice_rng.integers(1, 7, size=8)
print(dice)
print(dice.shape)
```

运行输出

```
[6 4 5 6 4 5 6 2]
(8,)
```

integers(1, 7) 默认包含 1、不包含 7；size=8 生成 8 次结果。种子 7 用于复现课堂实验，不表示掷出 7。

连续抽样与重新创建同种子的生成器有什么区别?

```
seed_rng = np.random.default_rng(7)
print(seed_rng.integers(1, 7, size=4))
print(seed_rng.integers(1, 7, size=4))
seed_rng = np.random.default_rng(7)
print(seed_rng.integers(1, 7, size=4))
```

运行输出

```
[6 4 5 6]
[4 5 6 2]
[6 4 5 6]
```

连续调用会推进生成器状态；重新创建相同种子的生成器会从头开始。在相同环境、相同调用顺序下可复现，不应在每轮模拟中反复重置种子。

从给定取值中抽样：左右移动概率不相同时，怎样表达？

```
choice_rng = np.random.default_rng(7)
sample_steps = choice_rng.choice([-1, 1], size=8,
                                 p=[0.3, 0.7])
print(sample_steps)
print((sample_steps == 1).mean())
```

运行输出

```
[ 1  1  1 -1  1  1 -1  1]
0.75
```

p 按取值顺序指定概率，合计为 1；默认有放回抽样，取值可以重复。向右概率 70% 不保证这 8 次中恰好 70% 向右。

均匀分布可模拟 0 到 10 分钟的等待；正态分布可作为围绕中心波动的简化模型。

```
continuous_rng = np.random.default_rng(7)
waits = continuous_rng.uniform(0, 10, size=5)
values = continuous_rng.normal(loc=0, scale=1, size=5)
print(np.round(waits, 2))
print(np.round(values, 2))
```

运行输出

```
[6.25 8.97 7.76 2.25 3.  ]
[-0.99  0.06  1.34 -0.49 -0.62]
```

uniform 指定区间；normal 的 loc 是均值，scale 是标准差，不是方差。正态分布没有固定上下界；round 这里只用于展示。

正态分布设置均值为 0，为什么抽出的数据均值不是恰好 0?

```
sample_rng = np.random.default_rng(7)
normal_sample = sample_rng.normal(0, 1, size=10000)
for count in [10, 100, 10000]:
    sample = normal_sample[:count]
    print(count, round(sample.mean(), 3),
          round(sample.std(), 3))
```

运行输出

```
10 -0.202 0.645
100 -0.173 0.875
10000 -0.012 0.994
```

每行依次是样本量、样本均值、样本标准差。样本量增大通常使统计量更稳定，但误差不保证每一步都缩小。std 默认 ddof=0；样本标准差常用 ddof=1。

每步向左或向右移动一格，起点为 0。先生成步长，再用 cumsum 得到位置；随机种子使课堂输出可复现。

```
walk_rng = np.random.default_rng(7)
directions = walk_rng.integers(0, 2, size=12)
steps = np.where(directions == 0, -1, 1)
positions = np.concatenate(([0], steps.cumsum()))
print(steps)
print(positions)
print("最远距离: ", np.abs(positions).max())
```

改成 100 步，并计算走动后位于起点右侧的比例；分母是否包括起点？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

先看 3 次实验、每次 4 步。每行是一个人的完整实验，每列是一步，单位为格。

```
many_rng = np.random.default_rng(7)
walk_steps = many_rng.choice([-1, 1], size=(3, 4))
walk_positions = walk_steps.cumsum(axis=1)
print(walk_steps)
print(walk_positions)
```

运行输出

```
[[ 1  1  1  1]
 [ 1  1  1 -1]
 [-1 -1 -1  1]]
[[ 1  2  3  4]
 [ 1  2  3  2]
 [-1 -2 -3 -2]]
```

axis=1 沿每行的时间方向累加。位置矩阵未加入起点；第一列是走完第 1 步的位置，最后一列是终点。

如果 1000 个人各走 10 步，有多少人回到原点、停在右侧？每一步独立且左右等概率。

```
many_rng = np.random.default_rng(7)
walk_steps = many_rng.choice([-1, 1], size=(1000, 10))
walk_positions = walk_steps.cumsum(axis=1)
endpoints = walk_positions[:, -1]
print(walk_positions.shape)
print("回到原点: ", (endpoints == 0).mean())
print("停在右侧: ", (endpoints > 0).mean())
```

运行输出

```
(1000, 10)
回到原点:  0.24
停在右侧:  0.359
```

分母是 1000 次实验，不是 10 步。频率是概率的模拟估计；左右对称仍有回到原点的可能，因此停在右侧的概率小于一半。

练习：步数和移动概率改变后会怎样？

先预测：走 11 步能回到原点吗？再模拟 1000 次各走 10 步，向右概率改为 70%，估计停在右侧的比例。

动手检查

先预测结果，再运行验证。参考答案见 Notebook 的折叠单元格。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. 随机数与重复模拟
8. 综合案例与小结

```python
rng = np.random.default_rng(8)            # 随机数生成器(固定种子)
n = 250                                    # 250个交易日

# 模拟日对数收益率：均值0.04%，标准差1.5%
rets = rng.normal(loc=0.0004, scale=0.015, size=n)

# 由对数收益率还原价格曲线，包含期初100元
prices = 100 * np.exp(np.concatenate(([0], rets.cumsum())))

print(f"日均对数收益率：{rets.mean():.4%}")
print(f"日波动率：   {rets.std():.4%}")
print(f"年化波动率：{rets.std()*np.sqrt(252):.2%}")
print(f"最高价/最低价：{prices.max():.2f} / {prices.min():.2f}")
```

- **年化波动率 = 日标准差** $\times\sqrt{252}$，此处采用独立同分布的日收益假设。
- 用到的全部是本章工具：向量化、cumsum、exp、聚合函数。

```python
# 上涨日占比
print(f"上涨日占比: {(rets > 0).mean():.2%}")

# 简单收益率低于-3%的极端日
simple_rets = np.exp(rets) - 1
extreme = simple_rets[simple_rets < -0.03]
print(f"极端下跌天数: {len(extreme)}")

# 最大回撤：包含期初价格的历史最高价 vs 当前价
peak = np.maximum.accumulate(prices)
drawdown = prices / peak - 1
print(f"最大回撤: {drawdown.min():.2%}")
```

- `np.maximum.accumulate`：到每个时点为止的历史最高价。
- **最大回撤**是基金与策略最常用的风险指标之一。
- 整段分析没有写一个 `for` 循环。

1. 创建与索引：说明每行、每列的含义，排序后保持字段配对。
2. 向量化与统计：检查缺失值，用广播、axis 和权重完成计算。
3. 随机数：明确分布参数、种子与样本形状，区分理论参数和样本结果。
4. 重复实验：每行一次实验，提取终点，用布尔均值估计事件概率。

模拟的边界

股价案例采用独立同分布的正态对数收益率，只用于教学，不是市场预测模型。年化波动率的平方根换算依赖相应假设。

- **创建**：`np.array`、`zeros/ones`、`arange/linspace`；属性 `shape/dtype`。
- **索引切片**：与列表同规则；二维写 `[行，列]`，`:` 表示全部。
- **向量化**：数组运算逐元素进行，一行顶循环，速度快百倍；
  数组的 `+` 是相加不是拼接。
- **布尔索引**：`a[a>0]` 筛选；`np.sum(条件)` 计数；条件组合用 `&`、`|`。
- **统计**：`mean/std/max/min`；`axis` 是被压掉的维度。
- **金融工具箱**：收益率 `p[1:]/p[:-1]-1`、年化波动率 $\sigma \times \sqrt{252}$、`cumsum/diff`、`np.where` 信号。

课后任务

1. 运行课堂 notebook，修改参数观察输出变化。
2. 给定价格数组 `np.array([50, 52, 51, 55, 53, 56])`：
   计算日收益率、找出最大单日涨幅、统计上涨天数。
3. 选做：用 `np.where` 构造信号"收益为正买 1 手、为负卖 1 手"，
   并计算该策略的每日收益（信号 × 次日收益，注意错位）。

下章预告

NumPy 数组采用统一类型；真实行情还需要日期、列名与表格对齐。
下一章学习 **pandas**：带标签的表格数据，金融数据分析的主力工具。

本章内容改编自教材第 4 章[1]。

- Wes McKinney, *Python for Data Analysis*, 3rd ed.
- `https://wesmckinney.com/book/numpy-basics`
- 新增中文说明、合成数据与练习为本课程自行编写。
- 阅读正文理解概念；在配套 Notebook 中运行、修改、核对。

[1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京 : 人民邮电出版社, 2018.



Thanks for your attention!

Q & A